

Using Open Source AI Models with Hugging Face

Hugging Face is a powerful platform that offers a wide range of open-source AI models and tools. In this project, I explored the Hugging Face ecosystem, focusing on the transformers library, and learned how to use pre-trained models for various tasks such as text classification, translation, and image captioning. Below, I'll walk you through each task I completed, along with the code and explanations.

Task 0: Setup

Before starting, I needed to install and import the necessary Python packages. These include torch, transformers, datasets, and huggingface_hub. The sentencepiece package is also required but doesn't need to be explicitly imported.

The sentencepiece package is required by transformers to perform inference with some of the pre-trained open source models on Hugging Face Hub and does not need to be explicitly imported.

```
!pip install datasets==2.13
!pip install huggingface_hub==0.16.4
!pip install pyarrow>=8.0.0
```

Hidden output

```
# Import os
import os

# Import torch
import torch

# Import huggingface_hub using the alias hf_hub
import huggingface_hub as hf_hub

# Import datasets
import datasets

# Import transformers
import transformers
```

Task 1: Download Pre-trained Models from the HF Hub

The Hugging Face Hub is a Git-based platform where users can store and share pre-trained models. In this task, I learned how to download and load a pre-trained model using the AutoTokenizer and AutoModel classes.

```
# Import the AutoTokenizer and AutoModel classes from transformers
from transformers import AutoTokenizer, AutoModel

# Load the pre-trained tokenizer of the "cardiffnlp/twitter-roberta-base-emoji" model
tokenizer = AutoTokenizer.from_pretrained("cardiffnlp/twitter-roberta-base-emoji")
```

Hidden output

What from_pretrained() does

The `from_pretrained()` method first searches for a model repository with the same name on the Hugging Face Hub but it also accepts a local path or a URL with the expected folder structure.

```
# Print the tokenizer to see the data preprocessing configuration for this model
print(tokenizer)
```

```
RobertaTokenizerFast(name_or_path='cardiffnlp/twitter-roberta-base-emoji', vocab_size=50265,
model_max_length=1000000000000000019884624838656, is_fast=True, padding_side='right',
truncation_side='right', special_tokens={'bos_token': '<s>', 'eos_token': '</s>',
'unk_token': '<unk>', 'sep_token': '</s>', 'pad_token': '<pad>', 'cls_token': '<s>',
'mask_token': AddedToken("<mask>", rstrip=False, lstrip=True, single_word=False,
normalized=False)}, clean_up_tokenization_spaces=True)
```

Tokenizers

NLP models can't process text inputs as it is and need to be converted to a fixed length mathematical format. The `Tokenizer` classes preprocess the text such that each word and punctuation is given a unique ID or a **token**, short sentences are padded and long sentences are truncated to create fixed-size input vectors. They also allow using additional tokens such as *bos*, *eos*, *unk*, *sep*, and more to specify start and end of sentences, and to assign token IDs to unknown words that are not in the tokenizer vocabulary.

The output of the tokenizer tells us this pretrained model uses a tokenizer with a 50265 unique token IDs that applies padding or truncation to the end of each input text, and removes leading (leftside) extra whitespaces.

```
# Load the pre-trained "cardiffnlp/twitter-roberta-base-emoji" model
model = AutoModel.from_pretrained("cardiffnlp/twitter-roberta-base-emoji")
```

Hidden output

Notes on Model Loading

I was only able to load a chunk of the model parameters included in the checkpoint with the `AutoModel` class.

This is because all `transformers` models are designed to have a single base class and multiple task-specific prediction classes built on top of it. The `AutoModel` class is designed to only load the base model parameters such as `RobertaModel` but not task-specific models such as `RobertaForSequenceClassification`.

```
# Get the model configuration
config = model.config
```

```
# Print model config's architectures attribute to see the model class name
print(model.config.architectures)
```

```
['RobertaForSequenceClassification']
```

I found out that the pretrained model that I'm trying to load is an instance of the `RobertaForSequenceClassification` class.

Task 2: Downstream Tasks and Task-Specific `AutoModel` Classes

In this task, I learned that the `AutoModel` class only loads the base model. To load task-specific models (e.g., for text classification), I used the `AutoModelForSequenceClassification` class.

```
# Load the correct AutoModel class for RobertaForSequenceClassification
from transformers import AutoModelForSequenceClassification

# Load the pretrained "cardiffnlp/twitter-roberta-base-emoji" model using the
AutoModelForSequenceClassification class and from_pretrained method
model = AutoModelForSequenceClassification.from_pretrained("cardiffnlp/twitter-roberta-base-emoji")
```

Hidden output

`AutoModelForSequenceClassification`: This class is used for models that perform sequence classification tasks, such as sentiment analysis or emoji prediction.

Task 3: Explicit Model and Preprocessor Class Names

Instead of using the Auto classes, I learned how to explicitly import and use the specific tokenizer and model classes.

```
# Import the explicit tokenizer class of "cardiffnlp/twitter-roberta-base-emoji"
from transformers import RobertaTokenizerFast

# Import the explicit model class of "cardiffnlp/twitter-roberta-base-emoji"
from transformers import RobertaForSequenceClassification
```

Hidden output

We can now use the explicit tokenizer and model classes the same way we use the Auto classes.

```
# Load the tokenizer and model of "cardiffnlp/twitter-roberta-base-emoji"
tokenizer = RobertaTokenizerFast.from_pretrained("cardiffnlp/twitter-roberta-base-emoji")
model = RobertaForSequenceClassification.from_pretrained("cardiffnlp/twitter-roberta-base-emoji")
```

Hidden output

Task 4: Creating an NLP Pipeline - Part 1/2

In this task, I created a language translation pipeline using the google/flan-t5-base model. This model can translate text between multiple languages.


```
# Import AutoModelForConditionalGeneration from transformers
from transformers import AutoModelForSeq2SeqLM

# Load the pre-trained google/flan-t5-base model and tokenizer
tokenizer = AutoTokenizer.from_pretrained("google/flan-t5-base")
model = AutoModelForSeq2SeqLM.from_pretrained("google/flan-t5-base")
```

Hidden output

Now it is possible to perform inference with the pretrained model. I started by defining an input sentence where I started by which language I would like to translate from and to, and add the sentence to be translated.

```
# Specify source and target languages for translation
source_lang = "English"
target_lang = "German"
input_text = f"translate {source_lang} to {target_lang}: How old are you?"

# Use the tokenizer to preprocess input text, with return_tensors="pt" to return PyTorch tensors
inputs = tokenizer(input_text, return_tensors="pt")

# Print the key and values of the preprocessed inputs dictionary
for key, value in inputs.items():
    print(key, value)

input_ids tensor([[13959, 1566, 12, 2968, 10, 571, 625, 33, 25, 58,
1]])
attention_mask tensor([[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]])
```

Task 5: Creating an NLP Pipeline - Part 2/2

In this task, I performed inference using the preprocessed input and decoded the model's output into human-readable text.

```
# Use torch.no_grad to prevent gradient accumulation - speeds up inference, reduces memory
with torch.no_grad():
    # Use model.generate method to perform inference
    # Use **inputs to pass the unpacked dictionary as multiple arguments to the generate
    method
    outputs = model.generate(**inputs)

# The output is a list of length batch_size - number of input texts - which is 1 in this case
# Print the length of outputs list
print(len(outputs))
```

1

The model output is a sequence of token IDs, where each token represents a unique word or punctuation in the vocabulary. To decode it to a human readable format, all I had to do is to map each token ID to its corresponding word. All `Tokenizer` classes of `transformers` provides a convenient `decode` method to do this.

```
# Use the decode method of the tokenizer to convert the output to a human readable format
print(tokenizer.decode(outputs[0]))
```

<pad> Wie old sind Sie?</s>

`tokenizer.decode()`: This converts the output tokens back into a human-readable string

Task 6: Datasets Library

In this task, I learned how to use the datasets library to load and explore datasets from the Hugging Face Hub.

```
# Import load_dataset from datasets package
from datasets import load_dataset

# Use the load_dataset function to load the "adiri/fashion_image_caption-100"
dataset = load_dataset("adiri/fashion_image_caption-100")

# Print the dataset object
print(dataset)
```

Hidden output

- `load_dataset()`: This function downloads and loads a dataset from the Hugging Face Hub.
- `dataset["train"][0]`: This retrieves the first sample from the training split of the dataset.

Each dataset is a dictionary with train, and optionally val and test splits. Each split contains an iterable subset of the data with one or more keys that represent the features of the dataset defined by the dataset owner.

```
# Print the first data sample in the dataset - an image and a text caption pair
print(dataset["train"][0])

# Visualize the image of the first data sample
dataset["train"][0]["image"]

{'image': <PIL.PngImagePlugin.PngImageFile image mode=RGB size=384x384 at 0x7FC1E56EB580>,
 'text': 'A dress with Color: Yellow, Department: Dresses, Detail: Zipper, Fabric-Elasticity: Medium Stretch, Fit: Fitted, Hemline: Pencil, Material: Gabardine, Neckline: Gathered, Pattern: Solid, Sleeve-Length: Sleeveless, Style: Party, Type: Bodycon Skirt, Waistline: Regular'}
```



You can also opt to download only a specific split of the dataset or pass in the `streaming=True` argument in order to download smaller subsets.

```
# Download only the training split of the dataset
dataset = load_dataset("adirik/fashion_image_caption-100", split="train")
```

Task 7: CV / Multi-Modal Pipeline

In this task, I created an image captioning pipeline using the Salesforce/blip-image-captioning-base model.

```
# Import BlipProcessor and BlipForConditionalGeneration from transformers
from transformers import BlipProcessor, BlipForConditionalGeneration

# Load preprocessor of "Salesforce/blip-image-captioning-base"
preprocessor = BlipProcessor.from_pretrained("Salesforce/blip-image-captioning-base")

# Load pre-trained "Salesforce/blip-image-captioning-base" model
model = BlipForConditionalGeneration.from_pretrained("Salesforce/blip-image-captioning-base")
```

Hidden output

- BlipProcessor: This preprocesses the image for the BLIP model.
- BlipForConditionalGeneration: This generates captions for the input image.

```
# Preprocess the first image of the dataset
inputs = preprocessor(dataset[0]["image"], return_tensors="pt")
```

The inference is almost the same as the NLP task as BLIP is a conditional text generation model. I needed to call `model.generate()` to run the model and use the `decode` method of the preprocessor to convert the output caption to a human readable format.

```
# With torch.no_grad()
with torch.no_grad():
    # Predict caption using the generate method of the model
    outputs = model.generate(**inputs)

# Decode model output to text
caption = preprocessor.decode(outputs[0], skip_special_tokens=True)

# Print decoded caption
print(caption)
```

a woman wearing a yellow sequin dress

This project was a great opportunity to explore the Hugging Face ecosystem and learn how to use pre-trained models for various tasks. I hope this breakdown helps you understand the steps and code involved!